

MN12832L VFD 드라이버 어떻게 더 안전하고 믿을 만하게 바꿨을까?

전광판을 움직이는 오래된 코드를 검사해 보니, 그림이 종이 밖으로 나가고, 비상 정지가 제대로 작동하지 않으며, 컴파일 방법에 따라 기다리는 시간이 사라지는 문제가 있었습니다. 문제를 먼저 재현한 뒤, 테스트와 정적분석, 서브에이전트 검토를 반복해 고쳤습니다.

대상: GU128x32D / MN12832L

작업 범위: STM32 소스 하드닝

설명 수준: 중학생 눈높이

한 줄 결론

컴퓨터 안에서 확인할 수 있는 문제는 고쳤고 모든 검사를 통과했습니다. 하지만 실제 보드에 꽂기 전에는 전압·극성·타이밍을 계측기로 다시 확인해야 합니다.

43

검사한 전체 화면 스캔 단계

240

한 단계에서 보내는 비트 수

11

변하지 않았음을 확인한 원본 파일

이 코드는 작은 전광판의 “그림 배달부”입니다

컴퓨터가 만든 그림을 아주 빠르게 잘라서 VFD 모듈로 전달합니다.

비유: 학교 운동장의 전광판을 한 번에 전부 켜는 대신, 가로줄을 아주 빠르게 차례로 켜면 사람 눈에는 전체 그림이 계속 보입니다. 이 과정을 **스캔**이라고 합니다.



임시 그림판은 어떻게 생겼나?

`data[4][128]` 이라는 상자에 그림을 저장합니다.

- ✓ 가로 128칸
- ✓ 세로는 8칸짜리 묶음 4개 = 32칸
- ✓ 점 하나는 켜짐(1) 또는 꺼짐(0)

공책 한 장을 세로 8줄씩 네 구역으로 나눈 모습과 같습니다.

모듈과 연결하는 신호

신호	쉬운 뜻	STM32 핀
SIN	0과 1을 보내는 선	PA1
CLK	다음 비트 알림	PA2
LAT	받은 내용을 화면에 적용	PA3
BLK	화면을 잠시 가림	PA4
EF / HV	전원 계통 켜기 제어	PFO / PF1

중요: 여기서 **HV** 는 높은 전압과 관련된 제어 신호입니다. 이름만 믿고 연결하면 안 되며, 실제 보드에서는 반드시 회로도나 계측기로 확인해야 합니다.

코드 안에 어떤 문제가 숨어 있었나?

당장 눈에 보이지 않아도 오래 켜 두거나 최적화하면 문제가 될 수 있었습니다.

매우 중요

1. 그림이 종이 밖으로 나감

글자를 화면 아래쪽에 그리면 `data[4]` 처럼 존재하지 않는 칸에 쓸 수 있었습니다.

공책 4쪽만 있는데 5쪽에 답을 쓰려는 것과 같습니다. 옆의 다른 자료까지 망가질 수 있습니다.

매우 중요

2. 비상 정지 장치가 부족함

코드가 멈추거나 큰 오류가 나도 화면 가림과 전원 제어 신호가 안전 상태로 바뀐다는 보장이 없었습니다.

기계가 고장 났는데 비상 정지 버튼이 모터와 연결되지 않은 상황입니다.

매우 중요

3. 기다리기 코드가 사라짐

빈 반복문으로 만든 시간 지연은 컴파일러 최적화를 켜면 아무 일도 하지 않는 코드로 지워졌습니다.

“80까지 세고 출발”이라고 적었지만, 똑똑한 번역기가 쓸모없다며 문장을 삭제한 셈입니다.

중요

4. 타이머 알림을 놓칠 수 있음

타이머와 본 프로그램이 같은 알림 메모를 동시에 만지면서 새 알림을 지워 버릴 가능성이 있었습니다.

선생님이 새 쪽지를 붙이는 순간 학생이 게시판을 지우는 상황입니다.

중요

5. 비슷한 코드가 두 벌

같은 이름의 글자 그리기 함수가 중복되어 함께 빌드하면 “이름이 두 개”라는 오류가 났습니다.

한 반에 학번 12번 학생이 두 명이라 출석을 부를 수 없는 것과 같습니다.

중요

6. 테스트와 실제 길이 달랐음

새로 만든 안전한 스캔 계산기는 테스트했지만, 실제 `main.c` 는 예전 계산을 계속 사용하고 있었습니다.

연습용 다리는 안전 검사를 통과했지만 실제 차는 낡은 다리로 가고 있었습니다.

추가 발견: 외부 글꼴 칩을 읽는 SPI 코드는 인자를 무시하고 반환값도 없었으며, STM32F4용으로 보이는 불완전한 파일이 STM32F0 코드와 섞여 있었습니다.

바로 고치지 않고, 먼저 문제를 잡는 그물을 만들었습니다

“고친 것처럼 보임”이 아니라 다시 실행해도 증명되는 방식으로 작업했습니다.

1 원본 보존 다운로드 풀더는 건드리지 않고 복사본과 SHA-256 지문을 만들었습니다.	2 문제 재현 경계 밖 쓰기와 사라지는 지연 코드를 실제 검사에서 실패하게 만들었습니다.	3 작게 분리 그림 계산, 비트 전송, 전원 상태를 각각 시험 가능한 부품으로 나눴습니다.	4 한 가지씩 수정 실패하던 테스트가 통과하도록 최소 범위로 코드를 바꿨습니다.
5 정적분석 실행하지 않아도 찾을 수 있는 위험 패턴과 컴파일 경고를 검사했습니다.	6 전체 회귀 테스트 새 수정이 예전 동작을 망가뜨리지 않았는지 전부 다시 돌렸습니다.	7 서브에이전트 검토 테스트 담당과 설계 담당이 독립적으로 빠진 문제를 찾았습니다.	8 승인될 때까지 반복 시작 순서 문제를 추가로 고친 뒤 최종 PASS를 받았습니다.

도구를 쉽게 설명하면

정적분석 = 코드 맞춤법 검사기

프로그램을 실제로 켜지 않아도 위험한 표현, 잘못된 반환, 비밀키 같은 흔적을 찾아봅니다.

- ✓ Clang 경고와 Static Analyzer
- ✓ Semgrep 위험 패턴 검사
- ✓ TruffleHog 비밀정보 검사

도구를 쉽게 설명하면

UBSan = 규칙 위반 감시 카메라

배열 밖 접근처럼 C언어에서 해서는 안 되는 동작이 실행되는 순간 바로 알려 줍니다.

- ✓ 화면 오른쪽·아래쪽 경계
- ✓ 정렬·비정렬 글자 그리기
- ✓ NULL과 잘못된 입력

핵심 원칙: 테스트가 빨간색(실패)인 것을 먼저 확인하고, 수정 뒤 초록색(통과)으로 바뀌는 것을 확인했습니다. 그래서 “원래부터 통과하던 가짜 테스트”를 피했습니다.

그림과 스캔 계산을 안전한 부품으로 바꿨습니다

복잡한 반복문을 작은 함수로 나누고, 가능한 모든 줄과 단계를 검사했습니다.

작업 전: 경계 확인이 부족

글자가 화면 아래나 오른쪽을 넘어가도 계속 쓰려고 했습니다.

```
y = 32
→ 다섯 번째 묶음 data[4] 접근
→ 다른 메모리를 망가뜨릴 가능성
```



작업 후: 화면 안에서만 그리기

좌표와 글꼴 주소를 먼저 확인하고, 화면 밖 부분은 잘라 냅니다.

```
x < 128, y < 32 확인
→ 보이는 부분만 저장
→ UBSan 경계 체크 통과
```

개선 1

스캔 포장기

vfd_scan_pack_step() 이 한 단계에 필요한 240비트를 정확히 만듭니다.

192 그림 비트 + 48 위치 비트

개선 2

비트 배달기

vfd_scan_emit_frame() 이 가장 높은 비트부터 순서대로 전송합니다.

192번째 뒤에 필요한 대기 1회

개선 3

상태 관리자

1단계부터 43단계까지 이동하고 기존 HV 펄스 상태를 따로 관리합니다.

복잡한 전역변수 감소

43단계 × 32줄을 어떻게 확인했나?

각 단계의 각 줄에 점을 하나씩 켜고, 그 점이 여섯 개 통로 중 정해진 통로로만 나오는지 확인했습니다. 마지막 43단계에는 실제로 두 열만 있으므로, 존재하지 않는 세 번째 열은 항상 0인지도 검사했습니다.

검사 항목	기대 결과	결과
홀수 단계 통로 순서	b1, 0, b2, 0, b3, 0	통과
짝수 단계 통로 순서	0, b3, 0, b2, 0, b1	통과
43번째 마지막 단계	126·127열만 사용	통과
위치 선택	이웃한 grid 두 개만 켜짐	통과

개선 효과: 이제 테스트에서 확인한 스캔 부품을 실제 main.c 도 그대로 사용합니다. 연습용 코드와 실제 코드가 서로 달랐던 문제가 사라졌습니다.

고장 났을 때도 안전한 순서로 멈추게 했습니다

전원이 관련된 코드는 정상 동작보다 “문제 발생 시 행동”이 더 중요합니다.

시작 순서

안전 핀을 먼저 준비

1. HAL_Init
2. GPIO 안전값 설정
3. 시스템 클럭 설정
4. 타이머 시작

기계를 켜기 전에 비상 정지 버튼부터 연결하는 순서입니다.

비상 정지

한 곳에서 같은 방식으로 끄기

1. BLK high : 먼저 화면 가리기
2. HV low : 높은 전압 제어 끄기
3. EF low : 나머지 전원 제어 끄기

문을 닫고, 큰 전원을 끄고, 보조 전원을 끄는 비상 절차입니다.

작업 전: 타이머 메모

ResetSCAN 을 타이머와 본 프로그램이 그냥 함께 사용했습니다.

타이머 “새 화면” 기록
분해 확인 후 0으로 리셋
→ 같은 순서로 새 화면을 리셋



작업 후: 잠깐 문 잠그기

volatile 표시와 아주 짧은 인터럽트 잠금으로 안전하게 가져옵니다.

문 잠금 → 열림 확인 대기
→ 안전 잠금 상태 복구
→ 서로 동행에 수정치 않음

처음 화면도 0으로 준비

전원을 켜자마자 쓰레기 값이 잠깐 보이지 않도록, 모든 비트가 0인 프레임을 먼저 보내고 화면을 가린 상태에서 latch합니다.

지연 코드가 사라지지 않게

각 반복에 실제 nop 명령을 넣고, Cortex-M0 최적화 결과에 반복 점프가 남는지 자동으로 확인합니다.

아직 계측이 필요한 이유: 반복문이 사라지지 않는 것은 증명했지만, 이름에 적힌 $13\mu s \sim 17\mu s$ 등이 실제 보드에서 정확히 그 시간인지는 MCU 종류와 클럭에 따라 달라집니다.

오류가 발생하면 모두 같은 비상 정지 사용

Error Handler

일반 초기화 실패

NMI

아주 중요한 인터럽트

HardFault

CPU의 심각한 오류

“통과”라는 말을 어떻게 확인했나?

서로 다른 검사와 독립 검토를 겹쳐서 한 검사의 빈틈을 다른 검사가 잡게 했습니다.

PASS UBSan 폰트 테스트	PASS 43×32 스캔 테스트	0 Qty 발견 문제	11/11 원본 해시 일치
-----------------------------	-----------------------------	-----------------------	--------------------------

검사	무엇을 찾나?	결과
UBSan 실행 테스트	배열 밖 접근과 잘못된 연산	통과
-Werror 컴파일	경고를 오류로 취급해 수상한 코드 차단	통과
Clang Static Analyzer	실행 경로 속 잠재 오류	통과
Semgrep	금지된 SPI 경로·빈 지연·위험 함수	0건
TruffleHog	코드 속 비밀번호·비밀키	0건
ARM 어셈블리 검사	최적화 뒤에도 지연 루프가 남는지	통과
소스 계약 검사	시작 순서·비상 정지·실제 packer 연결	통과
SHA-256 확인	다운로드 원본이 바뀌지 않았는지	11개 전부 OK

서브에이전트 1: 테스트 담당

테스트한 계산기가 실제 코드에 연결되지 않은 점, 헤더를 바꿔도 테스트가 다시 빌드되지 않는 점 등을 찾아냈습니다.

지적사항 반영 완료

서브에이전트 2: 설계·안전 담당

클록 설정이 GPIO 안전 준비보다 먼저라서, 시작 중 오류 시 안전 종료가 작동하지 않는 문제를 찾아냈습니다.

순서 수정 후 APPROVE

최종 독립 verifier: 커밋 4aff8b9 에서 모든 명령을 다시 실행하고, 저장소 상태·원본 해시·문서의 한계 설명까지 확인한 뒤 **PASS**를 판정했습니다.

무엇이 실제로 좋아졌나?

“코드가 예뻐졌다”가 아니라 실패 가능성과 확인 방법이 달라졌습니다.

항목	작업 전	작업 후
글자 그리기	화면 밖 메모리에 쓸 수 있음	좌표 확인과 clipping, UBSan 테스트
스캔 계산	main.c 에 긴 반복문	검증된 packer·emitter·state로 분리
타이머 알림	동시 수정 가능	volatile + 짧은 임계구역
비상 정지	오류 경로에서 안전 핀 보장 없음	BLK → HV → EF 순서로 공통 종료
시작 화면	shift register의 초기값이 불확실	0 프레임을 먼저 보내고 latch
지연 시간	최적화하면 반복문 삭제	실제 nop + ARM 코드생성 검사
중복 소스	같은 함수와 다른 MCU 코드 혼재	legacy/unsupported 로 격리
검증	재현 가능한 빌드·테스트 부족	한 명령으로 테스트·경고·분석 반복

현재 완료 판정

소스 하드닝과 호스트 검증 완료

- ✓ 컴퓨터에서 재현 가능한 테스트
- ✓ 정적분석과 경고 0건
- ✓ 독립 설계 검토 APPROVE
- ✓ 최종 verifier PASS

아직 완료가 아닌 것

- ! 실제 STM32용 완성 빌드
- ! 보드 핀 전압·극성 확인
- ! 오실로스코프로 타이밍 측정
- ! 장시간 전류·발열 시험

따라서: 지금 상태는 “안전하게 검토하고 다음 단계로 갈 수 있는 코드”입니다. “바로 연결해도 되는 완성 제품”이라는 뜻은 아닙니다.

재검증 명령

```
make clean all
PATH="$HOME/.qlty/bin:$PATH" QLTY_NO_UPDATE_CHECK=1 CI=1 qlty check --all --no-progress --summary
```


기억해야 할 핵심과 용어

어려운 단어보다 “왜 필요한가”를 이해하면 됩니다.

가장 중요한 세 문장

1. 문제를 고치기 전에 먼저 실패하는 테스트로 문제를 잡았습니다.
2. 정상 동작뿐 아니라 고장·시작·화면 경계 상황을 안전하게 만들었습니다.
3. 실제 하드웨어의 전압과 시간은 계속 전까지 “확인 필요”로 남겨 두었습니다.

프레임버퍼

화면에 그릴 점들을 잠시 저장하는 디지털 그림판

스캔

화면 일부를 빠르게 번갈아 켜 전체 그림처럼 보이게 하는 방법

비트

0 또는 1 하나. 전등으로 보면 꺼짐 또는 켜짐

클리핑

화면 밖으로 나간 그림 부분을 안전하게 잘라 내는 것

인터럽트

CPU가 하던 일을 잠깐 멈추고 급한 알림을 처리하는 기능

정적분석

프로그램을 실행하지 않고 코드 모양과 경로로 문제를 찾는 검사

UBSan

실행 중 C언어 규칙을 어기는 행동을 발견하는 검사 장치

Fail-safe

고장 나더라도 더 위험한 상태로 가지 않게 멈추는 설계

SHA-256

파일 내용으로 만든 긴 지문. 한 글자만 바뀌어도 지문이 달라짐

하드닝·호스트 검증

코드를 더 튼튼하게 만들고, 실제 보드 대신 개발 컴퓨터에서 확인하는 과정

산출물 위치

`/Users/pinion/dev/mn12832l-stm32-driver`

- ✓ `ANALYSIS.md` : 기술 분석
- ✓ `README.md` : 사용·검증 안내
- ✓ `reports/` : 이 HTML과 PDF

원본 보존

`/Users/pinion/Downloads/Plopydisk men`

- ✓ 원본 파일 수정 없음
- ✓ SHA-256 11개 모두 일치
- ✓ 작업은 별도 Git 저장소에서 진행

최종 한 줄 요약

넓은 전광판 코드를 “작동할 수도 있는 코드”에서 “무엇을 확인했고 무엇이 아직 남았는지 설명할 수 있는 코드”로 바꿨습니다.